PROMPT ENGINEERING

Comprehensive Training Guide

Zero-shot & Few-shot	Prompt Structure	Role & System Prompts
JSON & Schema Prompting	Prompt Evaluation	Hallucination Detection

Sigmoid Training Series • Enterprise AI Edition

Designed for Freshers & Early Practitioners

TABLE OF CONTENTS

CHAPTER 1

Prompt Basics

A **prompt** is any input — a question, instruction, or context snippet — you provide to a large language model (LLM) to elicit a desired response. Crafting effective prompts is the foundation of working productively with AI systems like GPT-4, Claude, Gemini, or open-source models.

1.1 What is a Prompt?

At the most basic level a prompt is text that guides the model's next token prediction. Behind the scenes the model treats the entire conversation — system instructions, prior turns, and the current user message — as a single tokenised sequence. The model predicts the most probable continuation of that sequence based on its training.

Example — Minimal Prompt

```
User: "Translate the following English text to French: Hello, how are you?"

Model: "Bonjour, comment allez-vous ?"
```

1.2 Anatomy of a Good Prompt

A well-crafted prompt typically contains some or all of the following elements:

Element	Purpose	Example
Instruction	Tells the model what to do	"Summarise the following article in 3 bullet points."
Context	Background that shapes the answer	"The user is a medical professional."
Input Data	The content to operate on	"Article: <paste text here>"
Output Format	Specifies structure / length	"Respond only with valid JSON."
Constraints	Guardrails / tone / scope	"Keep response under 100 words."

1.3 Key Prompt Properties

- **Clarity** — Avoid ambiguous phrasing. Use precise verbs: 'list', 'compare', 'explain step by step'.
- **Specificity** — The more specific the instruction the less room for off-target answers.
- **Conciseness** — Eliminate redundant words that add token cost without adding meaning.
- **Context** — Models have no memory between sessions; include all relevant context in the prompt.
- **Format Guidance** — If you need structured output, say so explicitly and give an example.
- **Iterability** — Treat prompts as living artefacts; version and improve them over time.

■ **TIP:** Start every prompt project with a one-sentence mission statement: 'This prompt will cause the model to…' This forces clarity before you write a single word.

CHAPTER 2

Zero-shot & Few-shot Prompting

Two of the most fundamental prompting strategies differ in how much task-specific guidance you provide inside the prompt itself — via worked examples called *shots*.

2.1 Zero-shot Prompting

In zero-shot prompting you provide **no examples**. You rely entirely on the model's pre-trained knowledge to generalise. This is the fastest approach and works well for straightforward tasks or when the instruction is self-explanatory.

Zero-shot — Sentiment Classification

```
Prompt:
  Classify the sentiment of the following review as
  Positive, Negative, or Neutral.

  Review: "The battery life is terrible but the camera is amazing."

Model Output:
  Mixed (leans Positive)
```

2.2 Few-shot Prompting

Few-shot prompting supplies **2–8 worked examples** (input → output pairs) before the actual query. This in-context learning helps the model infer the task format, tone, and output style without retraining.

Few-shot — Sentiment Classification (3-shot)

```
Prompt:
  Classify sentiment as Positive, Negative, or Neutral.

  Review: "Absolutely love this product!" → Positive
  Review: "Waste of money, broke after one day." → Negative
  Review: "Works as described, nothing special." → Neutral

  Review: "The battery life is terrible but the camera is amazing."

Model Output:
  Mixed / Neutral
```

2.3 Comparison & When to Use

Aspect	Zero-shot	Few-shot
Examples needed	None	2–8 representative pairs
Best for	General, well-known tasks	Custom formats, niche tasks
Token cost	Low	Higher (examples add tokens)
Consistency	Variable	More consistent
Risk	Model may guess format	Bad examples bias output
Iteration speed	Fast	Slower (curate examples)

■ **TIP:** Start zero-shot; add examples only when the model's output format is wrong or inconsistent. Three well-chosen examples usually outperform ten mediocre ones.

■■ **CAUTION:** Ensure few-shot examples cover edge cases (negatives, ambiguous inputs). Biased examples produce biased outputs.

CHAPTER 3

Prompt Structure

Prompt structure refers to the deliberate organisation of information within a prompt. A well-structured prompt reduces ambiguity, improves reproducibility, and makes debugging easier.

3.1 Instruction-Input-Output Pattern (IIO)

The IIO pattern separates three concerns clearly:

- **Instruction** — What the model should do.
- **Input** — The data or context to act on.
- **Output Hint** — Format, length, or structure expectations.

IIO Pattern Example

```
## Instruction
You are a professional email editor. Rewrite the email below to be
concise, professional, and free of grammatical errors.

## Input
Email: "hey can u send me the report asap its super urgent thanks"

## Output
Return only the rewritten email. No commentary.
```

3.2 Context, Constraints & Format Sections

For complex prompts, splitting into labelled sections using markdown headers (## **Context**, ## **Rules**, ## **Task**) dramatically improves model adherence, especially with longer context windows.

Sectioned Prompt Template

```
## Context
You are a customer-support bot for AcmeCorp, a SaaS analytics company.
Only answer questions about our product. Do not discuss competitors.

## Rules
- Be polite and professional.
- If unsure, say: "I will escalate this to a human agent."
- Never reveal internal pricing or roadmap details.

## Task
Answer the following customer query:
{query}

## Output Format
Plain text, max 3 sentences.
```

3.3 Chain-of-Thought (CoT) Structuring

Chain-of-Thought prompting asks the model to reason step-by-step before giving a final answer. This is especially powerful for arithmetic, logic, and multi-step reasoning tasks.

CoT Trigger Phrase

```
# Standard
Q: If a shirt costs $15 and is 20% off, what is the final price?
```

```
A: $12

# With CoT
Q: If a shirt costs $15 and is 20% off, what is the final price?
Think step by step, then give the final answer.

A: Step 1 — Calculate discount: 20% of $15 = $3.
   Step 2 — Subtract: $15 - $3 = $12.
   Final answer: $12.
```

■ **TIP:** Adding "Let's think step by step" or "Think step by step" to any reasoning prompt has been shown to improve accuracy by 20–40% on benchmark tasks.

CHAPTER 4

Role & Structured Prompting

Role prompting assigns a persona to the model, shaping its tone, vocabulary, and reasoning style. Structured prompting combines role assignment with a defined output schema.

4.1 Role Prompting

Prefixing a prompt with "You are a [role]..." conditions the model's responses without changing its weights. The model uses its training to emulate the expected behaviour of that role.

Role Prompting Examples

```
# Data Scientist Role
You are a senior data scientist at a Fortune 500 company.
Review the following Python code for efficiency and correctness.
Suggest improvements with explanations.

# Legal Reviewer Role
You are a contract attorney specialising in SaaS agreements.
Identify any clauses in the following contract that present risk
to the client. List each risk with a severity: High / Medium / Low.
```

4.2 Persona Design Guidelines

- **Be specific** — 'Senior DevOps engineer with 10 years of Kubernetes experience' > 'expert'.
- **Include expertise level** — Calibrates vocabulary and assumed audience knowledge.
- **Specify audience** — 'Explain to a non-technical CFO' adjusts depth automatically.
- **Add behavioural traits** — 'Be direct and concise. Avoid jargon.' shapes style.
- **Avoid contradictions** — 'Act as a strict editor but be encouraging' confuses the model.

4.3 Structured Output Prompting

Combining a role with explicit output structure (tables, numbered lists, XML tags) gives highly predictable, parseable responses — essential for downstream automation.

Structured Role Prompt

```
You are a technical documentation writer.
For each function described below, produce a structured doc entry.

Format each entry exactly as:
  FUNCTION: <name>
  PURPOSE:  <one sentence>
  PARAMS:   <param: type — description, ...>
  RETURNS:  <type — description>
  EXAMPLE:  <short code snippet>

Function: calculate_discount(price: float, pct: float) -> float
```

■ **NOTE:** Role prompts work best in the System message layer (see Chapter 5). Keep role instructions stable and move task-specific instructions to the User turn.

CHAPTER 5

System vs User Prompts

Modern LLM APIs expose at least two message roles: **system** and **user** (and often **assistant** for multi-turn history). Understanding the distinction is critical for building reliable, production-grade AI applications.

5.1 System Prompt

The system prompt is a privileged, persistent instruction layer that frames all subsequent user interactions. It is processed before user messages and typically contains:

- Role / persona definition
- Behavioural rules and guardrails
- Output format requirements
- Domain scope (what the model should/should not discuss)
- Security and safety instructions

System Prompt Example (OpenAI Chat API)

```
{
  "role": "system",
  "content": "You are HelpBot, a friendly customer support assistant
    for TechStore. Only answer questions about our products.
    If asked off-topic questions, politely redirect.
    Always respond in English. Keep answers under 150 words.
    Never reveal internal order data unless the user provides
    their verified order ID."
}
```

5.2 User Prompt

The user prompt is the dynamic, per-request input that changes with each interaction. Best practices for user prompts:

- Inject only user-supplied variables; keep structure in the system prompt.
- Sanitise user input before injection to prevent prompt injection attacks.
- Use clear delimiters (triple quotes, XML tags) when inserting user content.
- Keep user prompts focused on a single task per turn.

User Prompt with Safe Delimiter

```
{
  "role": "user",
  "content": "Summarise the article below in 3 bullet points.\n\n
    """\n
    {user_article_text}\n
    """
}
```

5.3 Layered Prompt Architecture

A production-ready prompt stack follows this hierarchy:

Layer	Role	Changes?	Responsibility
System Prompt	Developer-set	Rarely	Persona, rules, format, guardrails
Few-shot Examples	Developer-set	Versioned	Task demonstrations
Conversation History	Auto-managed	Each turn	Contextual memory
User Message	User-provided	Every turn	The actual query / input data

■■ **CAUTION:** Never put sensitive secrets (API keys, PII) in system prompts. LLMs can be prompted to reveal their system instructions via adversarial 'ignore previous instructions' attacks.

CHAPTER 6

JSON Output Prompting

Getting an LLM to return valid, parseable JSON is essential for integrating AI into pipelines, APIs, and databases. Several techniques make this reliable.

6.1 Instructing JSON Output

Three approaches — from weakest to strongest:

- **Instruction only** — Tell the model to respond in JSON. Works well for capable models.
- **Example-driven** — Show an example JSON structure the model should mirror.
- **JSON Mode / Structured Outputs API** — OpenAI, Anthropic, and others expose API flags that force JSON.

Prompt — JSON Instruction Only

```
Extract the following fields from the product description and return
ONLY valid JSON. Do not include any other text.

Fields: product_name, price_usd, in_stock (boolean), features (list of strings)

Product description:
"The ProX Headphones ($149.99) are currently available.
 Key features: noise cancellation, 30-hour battery, USB-C charging."
```

Expected JSON Output

```
{
  "product_name": "ProX Headphones",
  "price_usd": 149.99,
  "in_stock": true,
  "features": ["noise cancellation", "30-hour battery", "USB-C charging"]
}
```

6.2 Validation Strategies

- **Parse defensively** — Wrap json.loads() in try/except; retry on failure.
- **Schema validation** — Use jsonschema (Python) or Zod (TypeScript) to validate structure.
- **Model-level enforcement** — Use OpenAI's response_format: {type: 'json_object'} parameter.
- **Post-processing** — Strip markdown code fences (```json ... ```) before parsing.
- **Retry with correction** — On invalid JSON, re-send with the error message asking the model to fix it.

Python — Robust JSON Parsing with Retry

```
import json, re

def parse_json_response(text: str) -> dict:
    # Strip markdown code fences if present
    text = re.sub(r"```json|```", "", text).strip()
    try:
        return json.loads(text)
    except json.JSONDecodeError as e:
        raise ValueError(f"Model returned invalid JSON: {e}\n{text}")
```

CHAPTER 7

Schema-based Prompting

Schema-based prompting extends JSON output by providing the model with a formal schema definition — either as a JSON Schema object or a TypeScript interface — so it knows exactly what structure and types are expected.

7.1 Defining Output Schemas in the Prompt

Schema Embedded in Prompt

```
Return a JSON object that strictly matches the following schema.
Do not include any fields not listed in the schema.

Schema:
{
  "candidate_name": "string",
  "years_of_experience": "integer",
  "skills": ["string"],
  "suitable_for_role": "boolean",
  "rejection_reason": "string | null"
}

CV Text: {cv_text}
```

7.2 Using JSON Schema Format

JSON Schema is the industry standard for describing JSON data structures. Passing it directly in the prompt or via the API's structured output parameter gives the model the most precise guidance.

JSON Schema — Invoice Extraction

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["invoice_number", "vendor", "total_amount", "line_items"],
  "properties": {
    "invoice_number": { "type": "string" },
    "vendor":         { "type": "string" },
    "total_amount":   { "type": "number" },
    "line_items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "description": { "type": "string" },
          "quantity":    { "type": "integer" },
          "unit_price":  { "type": "number" }
        }
      }
    }
  }
}
```

7.3 TypeScript Interface Pattern

For teams using TypeScript or when prompting GPT-4 / Claude (which understand TypeScript well), a TypeScript interface can be more readable than JSON Schema.

TypeScript Interface in Prompt

```
Return a JSON object matching the TypeScript interface below.

interface ProductReview {
  reviewer_name: string;
  rating: 1 | 2 | 3 | 4 | 5;
  pros: string[];
  cons: string[];
  verdict: "buy" | "avoid" | "neutral";
  summary: string;   // max 50 words
}
```

■ **NOTE:** Always include required vs optional fields and value constraints (enums, min/max) in your schema. Models respect constraints when they're explicit.

CHAPTER 8

Prompt Evaluation

Prompt evaluation is the systematic process of measuring whether a prompt reliably produces outputs that meet quality, accuracy, safety, and format requirements. It is the engineering discipline that separates production-grade prompts from ad-hoc experiments.

8.1 Evaluation Metrics

Metric	Description	How to Measure
Accuracy	Factual correctness	Compare to ground truth labels
Format Adherence	Follows required structure	Parse output; check schema compliance
Completeness	All required elements present	Checklist of expected fields
Conciseness	No unnecessary verbosity	Word/token count vs target
Tone Consistency	Matches persona/style guide	Human rater rubric or LLM-as-judge
Safety	No harmful or biased output	Red-team testing, safety classifiers
Latency	Response time / token cost	API timing + token count logs

8.2 Human vs Automated Evaluation

Dimension	Human Eval	Automated Eval
Accuracy	Gold standard	Good with reference answers
Subjectivity	Handles nuance	Struggles without rubric
Scale	Expensive, slow	Fast, cheap, scalable
Bias	Annotator variance	Model/metric bias
Best use	Final validation	Continuous regression testing

8.3 A/B Testing Prompts

Treat prompt versions like code — version-control them and run controlled tests:

- Hold all variables constant (model, temperature, dataset) except the prompt.
- Use a fixed, representative evaluation dataset (50–200 examples minimum).
- Score outputs with automated metrics and spot-check with humans.
- Use statistical significance tests before declaring a winner.
- Log prompt version, model version, and eval scores together.

Simple A/B Eval Script (Python pseudocode)

```
from eval_utils import score_accuracy, score_format

prompts = {"v1": PROMPT_V1, "v2": PROMPT_V2}
results = {}
```

```
for name, prompt in prompts.items():
    outputs = [llm(prompt.format(input=x)) for x in TEST_INPUTS]
    results[name] = {
        "accuracy": score_accuracy(outputs, GROUND_TRUTH),
        "format_pass_rate": score_format(outputs)
    }

print(results)  # Pick highest accuracy + format combo
```

CHAPTER 9

Prompt Debugging

When a prompt produces unexpected output, systematic debugging — not trial-and-error guessing — will find the root cause faster and produce a more robust fix.

9.1 Common Failure Modes

Failure Mode	Symptom
Format drift	Model ignores format instructions and returns prose instead of JSON/bullets.
Context bleed	Earlier conversation turns influence the answer in unintended ways.
Over-refusal	Safety guardrails trigger on benign prompts due to ambiguous phrasing.
Instruction conflict	System and user prompts contain contradictory rules.
Truncation	Output cuts off mid-sentence because max_tokens is too low.
Role confusion	Model breaks persona mid-response when asked meta-questions.
Hallucination	Model fabricates facts not present in provided context.
Language drift	Model switches language when user input is in a different language.

9.2 Prompt Debugging Checklist

- ■ Is the instruction **unambiguous**? (Read it as if you'd never seen the task.)
- ■ Does the prompt include **all necessary context**? (Model has no external memory.)
- ■ Are there **conflicting instructions** between system and user messages?
- ■ Is the **output format** explicitly specified with an example?
- ■ Is **max_tokens** high enough for the expected output length?
- ■ Is the **temperature** appropriate? (0 for deterministic, 0.7 for creative.)
- ■ Did you test with **multiple diverse inputs**, not just the happy path?
- ■ Are user inputs **sanitised** to prevent prompt injection?
- ■ Have you checked **token limits** — is the full prompt fitting in the context window?
- ■ Did you isolate the failure by **stripping the prompt to its minimum** and adding back?

9.3 Iterative Refinement Process

Debug Loop

```
1. OBSERVE   → What exactly is wrong with the output?
2. HYPOTHESISE → Which part of the prompt caused it?
3. ISOLATE   → Strip prompt to minimum. Reproduce the bug.
4. FIX       → Change ONE thing at a time.
5. TEST      → Run against full eval dataset, not just the failing example.
6. DOCUMENT  → Log what changed and why.
```

■ **TIP:** Ask the model to explain its reasoning: append 'Before answering, explain your interpretation of the instructions.' This often reveals where the model misunderstood.

CHAPTER 10

Hallucination Detection Basics

Hallucination is the phenomenon where a language model generates plausible-sounding but factually incorrect, unverifiable, or entirely fabricated information. Understanding and detecting hallucinations is critical before deploying LLMs in high-stakes domains.

10.1 Types of Hallucinations

Type	Description	Example
Factual Fabrication	Invents facts not in context	"Einstein won the Nobel Prize in 1925" (it was 1921)
Source Fabrication	Cites non-existent papers/URLs	Made-up journal reference with real-looking DOI
Entity Confusion	Mixes attributes of two similar entities	Attributes Tesla quote to Edison
Numeric Hallucination	Wrong dates, numbers, statistics	"GDP of $4.2T" when actual is $3.1T
Logical Inconsistency	Contradicts itself in the same response	States X then later states not-X
Instruction Drift	Ignores part of a long instruction	Skips a required output field silently

10.2 Detection Techniques

Prompt-level Techniques

- **Ask for sources** — 'Cite your sources.' Hallucinated sources are often detectable.
- **Confidence elicitation** — 'How confident are you? Rate 1–10.' Low scores flag uncertainty.
- **Contrastive probing** — Ask the same question differently; contradictory answers signal hallucination.
- **Self-consistency sampling** — Sample N responses at temperature > 0; majority vote filters noise.
- **Verification prompts** — 'Is the following statement true based only on the provided context? [statement]'

System-level Techniques

- **RAG grounding** — Retrieval-Augmented Generation anchors answers to verified document chunks.
- **Knowledge base cross-check** — Programmatically verify named entities and numbers against a DB.
- **LLM-as-judge** — A second LLM call evaluates whether the answer is supported by the context.
- **Perplexity monitoring** — Very low perplexity on obscure claims is a hallucination signal.
- **External fact-checking APIs** — Google Fact Check Tools, news APIs for claim verification.

10.3 Mitigation Strategies

Hallucination-Resistant Prompt Template

```
## Context (ground truth only)
{retrieved_document_chunks}

## Instruction
Answer the question below using ONLY information from the Context above.
If the answer is not contained in the Context, respond:
```

```
"I do not have enough information to answer this question."
Do NOT use any prior knowledge.

## Question
{user_question}
```

Strategy	Effect	Implementation Effort
System prompt grounding	High — anchors to verified docs	Low
Temperature = 0	Medium — reduces randomness	Trivial
RAG pipeline	High — factual retrieval	High
Self-consistency voting	Medium — filters outliers	Medium
Post-gen fact-check	High — catches errors	High

■ **TIP:** The single most effective mitigation is to provide the model with a relevant context document and instruct it to answer ONLY from that document. This converts an open-book exam into a closed-book reading comprehension task — dramatically reducing fabrication.

■■ **CAUTION:** Never deploy LLMs for medical diagnosis, legal advice, or financial decisions without a verified human review step. Hallucinations in these domains cause real-world harm.

QUICK REFERENCE CARD

Topic	Key Takeaway
Zero-shot	No examples needed; fast but less consistent
Few-shot	2-8 examples in prompt; curate for edge cases
IIO Structure	Instruction → Input → Output format in every prompt
Chain-of-Thought	Add 'Think step by step' for reasoning tasks
Role Prompting	Be specific: seniority + domain + audience
System Prompt	Persona + rules + format; rarely changes
User Prompt	Dynamic task input; sanitise before injection
JSON Output	Specify schema + strip markdown fences on parse
Schema Prompting	Embed JSON Schema or TypeScript interface directly
Evaluation	Accuracy + Format + Completeness + Safety metrics
Debugging	Isolate → Hypothesise → Change ONE thing → Retest
Hallucination	Ground in context; instruct 'answer only from above'